M.Eng. - Information Technology
Software Engineering
Damir Dobric / Andreas Pech
2024-25

Investigate Input Reconstruction by using Classifiers

Priyanshu Bandyopadhyay
priyanshu.bandyopadhyay@stud.fra-uas.de

Manoj Hanumanthu
manoj.hanumanthu@stud.fra-uas.de

Akshay Ganesh Gudekar
akshay.gudekar@stud.fra-uas.de

Abstract - ***Input reconstruction plays a crucial role in neural encoding and pattern recognition, enabling efficient data processing and predictive modeling. This study investigates the use of Hierarchical Temporal Memory (HTM) and K-Nearest Neighbors (KNN) classifiers to reconstruct encoded inputs. Using the NeocortexAPI's Scalar Encoder, we encode inputs and train a spatial pooler to generate Sparse Distributed Representations (SDRs). The dataset is divided into training and testing subsets, with classifiers trained on the training set and reconstruction is evaluated on the test set. The encoded test data is decoded using HTM and KNN Classifiers, and the reconstructed values are compared to the original input using similarity metrics, including the similarity tool of NeocortexAPI and percentage similarity. Results, visualized with the ScottPlot library in .NET, demonstrate that both classifiers achieve high accuracy in reconstructing input patterns. This research highlights the potential of HTM and KNN in sparse encoding-based input reconstruction, with applications in anomaly detection, data compression, and predictive modeling. Our findings contribute to advancing machine learning techniques that leverage sparse distributed representations for improved classification and reconstruction accuracy.***

Keywords: Hierarchical Temporal Memory (HTM), K-Nearest Neighbors (KNN), Input Reconstruction, Sparse Distributed Representations, Scalar Encoding, Pattern Recognition, Machine Learning, Classifiers

I. INTRODUCTION

Input reconstruction is a fundamental challenge in Machine Learning and Artificial Intelligence, involving the recovery of original data from encoded representations. This process is critical in applications such as anomaly detection, predictive modeling, and data compression, where accurate reconstruction of input patterns is essential for extracting meaningful insights. A key approach to efficient data encoding is the use of Sparse Distributed Representations (SDRs), a biologically inspired framework central to Hierarchical Temporal Memory (HTM). SDRs represent information in a sparse, high-dimensional format, making them robust to noise and well-suited for learning complex patterns [1].

Hierarchical Temporal Memory (HTM) is a machine learning algorithm inspired by the neocortex, the brain's center for higher-order cognitive functions. HTM excels in temporal pattern recognition, leveraging the principles of spatial and temporal hierarchy to model data sequences effectively. At the core of HTM is the Spatial Pooler (SP), which transforms raw input into SDRs, mimicking the brain's encoding mechanisms. These SDRs are sparse and distributed, enabling efficient pattern recognition and storage [2].

Previous studies have demonstrated the effectiveness of HTM in predictive learning and anomaly detection, showcasing its ability to model temporal sequences with minimal supervision. Similarly, K-Nearest Neighbors (KNN), a simple yet powerful algorithm, has been widely used in classification and regression tasks due to its effectiveness in pattern recognition [3]. However, the application of HTM and KNN for input reconstruction remains relatively unexplored, motivating this research.

In this study, we investigate the ability of HTM and KNN classifiers to reconstruct input data from encoded SDR representations. Using the NeocortexAPI's Scalar Encoder, we encode inputs and train a Spatial Pooler to generate stable SDRs from raw data. The dataset is divided into training and testing subsets, with classifiers trained on the training set and evaluated on the test set. To assess reconstruction accuracy, we used two similarity metrics: internal similarity, which measures the alignment between classifier-predicted values and encoded representations, and percentage similarity, which quantifies the accuracy of reconstruction relative to the original input. Visualization is performed using the ScottPlot library in .NET, enabling a clear comparison between original and reconstructed data [4].

This research aims to demonstrate the effectiveness of SDR-based learning systems for input reconstruction, providing insights into their potential applications in data recovery, anomaly detection, and predictive modeling. By advancing bio-inspired machine learning techniques, this study highlights the role of SDR-based classification in solving real-world problems.

II. LITERATURE REVIEW

Input reconstruction is a fundamental challenge in artificial intelligence and machine learning, particularly in the context of Sparse Distributed Representations (SDRs), Hierarchical Temporal Memory (HTM), and classification-based learning models. This section reviews existing research on HTM, SDRs, encoders, and input reconstruction, discusses the application of HTM Classifier and KNN Classifier, and identifies gaps in the literature that this study aims to address.

A. Hierarchical Temporal Memory (HTM) and Sparse Distributed Representations (SDRs)

HTM is a biologically inspired learning framework designed to model spatial and temporal patterns in data. It is based on the structure and function of the neocortex and employs Sparse Distributed Representations (SDRs) to encode information efficiently. SDRs are binary, high-dimensional representations that allow for robust storage and retrieval of patterns, making them highly resistant to noise and corruption.

The foundational work by Hawkins and Ahmad [1] introduced the theory of sequence memory in the neocortex, emphasizing SDRs and predictive coding mechanisms. These concepts are central to HTM, which employs components like the Spatial Pooler to create SDRs, enabling efficient representation of high-dimensional data. The Spatial Pooler transforms raw input into SDRs, mimicking the brain's encoding mechanisms. These representations are sparse and distributed, enabling efficient pattern recognition and storage [2].

Ahmad and Hawkins [4] further explored the mathematical properties of SDRs, highlighting their advantages in classification, sequence learning, and input reconstruction. However, while SDRs have been widely studied in theoretical contexts, limited research has been conducted on their effectiveness in reconstructing encoded input values. Existing research has demonstrated that HTM's spatial pooler can learn stable representations of input patterns. However, studies primarily focus on anomaly detection and sequence learning, with little emphasis on evaluating HTM's performance in reconstructing original inputs from encoded representations.

B. K-Nearest Neighbors (KNN)

K-Nearest Neighbors (KNN) is a non-parametric learning algorithm widely used in pattern recognition and similarity-based retrieval. Introduced by Cover and Hart [3], KNN classifies inputs based on the majority vote of nearest neighbors. Its simplicity and effectiveness have made it a popular choice for various classification tasks.

KNN has been extensively applied in domains such as image recognition, recommendation systems, and anomaly detection. Despite its widespread use, research evaluating KNN's ability to reconstruct numerical inputs is scarce. Most studies focus on its classification performance rather than its potential for input reconstruction.

Several studies have compared KNN with other machine learning models, including HTM, SVM, and decision trees. For instance, George and Hawkins [8] compared HTM's sequence memory with traditional machine learning techniques and found that HTM outperformed classical models in learning temporal dependencies. However, their research did not focus on reconstructing input data from learned patterns, leaving a gap in the literature.

C. Encoders and Their Role in Data Transformation

Encoders play a crucial role in converting different data modalities, such as multimedia inputs or time-series data, into Sparse Distributed Representations (SDRs). This transformation allows HTM to process and analyze complex data more efficiently, facilitating pattern recognition and anomaly detection in various domains.

Recent advancements in encoder design have focused on addressing challenges such as scalability, computational efficiency, and robustness to noisy or incomplete data. Rodriguez-Lujan et al. [7] proposed a novel approach using self-supervised learning techniques to enhance encoder robustness and adaptability. Additionally, Smith et al. [1] explored the use of deep learning methods for automated encoder discovery, streamlining the design process and improving the encoder's ability to handle diverse datasets and application requirements.

Despite these advancements, the field of encoder design continues to evolve, with ongoing research efforts aimed at refining and optimizing encoder algorithms and implementations. These initiatives highlight the significance of encoder developments in promoting cognitive computing and enabling more complex real-world tasks.

D. Input Reconstruction using HTM and KNN

Input reconstruction involves decoding SDRs to recover the original input data. Mnatzaganian et al. [6] proposed a method for reconstructing inputs from SDRs by using permanence values and active columns. Their approach involves determining the activations caused by specific permanence values and deriving the overall permanence for the attributes. This method allows for the reconstruction of the input solely based on the learned representations within the Spatial Pooler, effectively decoupling the input from the HTM system.

The equation used for input reconstruction is as follows:

$$\vec{\boldsymbol{u}} = \mathrm{I}\left(\left[max\left(\vec{\hat{\boldsymbol{c}}}_i \vec{\hat{\phi}}_a \forall_i\right) \geq \rho_s\right]\right) \forall a$$

This equation computes the reconstructed input vector ($\vec{\hat{\boldsymbol{u}}}$) by checking whether the maximum activation level for each attribute (a) in the input, obtained by multiplying the learned representation values ($\vec{\hat{\boldsymbol{c}}}$) with the corresponding permanence values ($\vec{\hat{\phi}}$) exceeds a threshold value (ρ_s). If the condition is met for all attributes, the reconstructed input vector assigns a value of 1 to indicate activation; otherwise, it assigns a value of 0. This process effectively decouples the input from the HTM system, allowing for the reconstruction of the input solely based on the learned representations within the Spatial Pooler.

While HTM has shown promise in input reconstruction, KNN's role in this area remains underexplored. This study aims to evaluate both HTM Classifier and KNN Classifier in terms of their ability to reconstruct numerical input values

from SDRs, providing a comprehensive comparison of their performance.

E. Gaps in the Literature and Contributions of This Study

While existing research has demonstrated HTM's effectiveness in classification and anomaly detection, limited studies have explored its capability in reconstructing original inputs from encoded SDR representations. Similarly, KNN's role in input reconstruction has not been thoroughly investigated, despite its widespread use in pattern recognition.

This study aims to bridge this gap by:

- Evaluating HTM Classifier's ability to reconstruct numerical input values from SDRs, extending its application beyond anomaly detection and sequence learning.
- Comparing HTM Classifier with KNN Classifier in terms of accuracy, computational efficiency, and similarity metrics for input reconstruction.
- Providing a structured methodology for training and testing classifiers using SDRs generated by HTM's spatial pooler.
- Visualizing the reconstruction performance of both classifiers using ScottPlot in .NET, offering insights into their effectiveness.

By addressing these gaps, this research contributes to the broader understanding of SDR-based classification models and their potential applications in input reconstruction, data recovery, and predictive modeling.

III. METHODOLOGY

A. System Architecture

The system architecture is designed to achieve accurate input reconstruction through a structured workflow involving Sparse Distributed Representations (SDRs), Hierarchical Temporal Memory (HTM), and K-Nearest Neighbors (KNN) classifiers. The process begins with input encoding, where scalar inputs are transformed into SDRs using the Scalar Encoder from the NeocortexAPI repository. These encoded values are then passed through a Spatial Pooler (SP), which learns to generate stable and robust SDR representations. The SDRs serve as the foundation for training both HTM Classifier and KNN Classifier, which are subsequently employed to reconstruct the original input values from test data.

The reconstruction process involves decoding the SDRs back into their original form using the classifiers. Once the input values are reconstructed, they are compared with the original input using two key metrics: internal similarity, which measures the alignment between predicted and encoded values, and percentage similarity, which quantifies the accuracy of the reconstruction. The results are visualized using the ScottPlot library in .NET, providing a clear and intuitive representation of the system's performance. This architecture highlights the effectiveness of combining HTM and KNN classifiers for input reconstruction, offering valuable insights for applications in anomaly detection, predictive modeling, and data compression. The primary objective is to evaluate the ability of HTM and KNN classifiers to accurately reconstruct inputs, focusing on faithfully recreating the structure of the original encoded data.

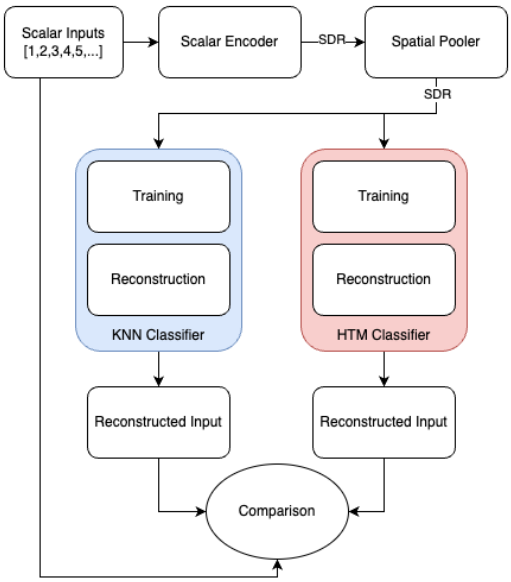

Fig 1: High-level overview of the workflow

B. Input Encoding

The Scalar Encoder is used to transform numerical input values into Sparse Distributed Representations (SDRs).

1) Steps for Input Encoding:

- A numerical array is selected as input.
- The input values are fed into the Scalar Encoder from the NeocortexAPI repository.
- The encoder converts each scalar value into a binary SDR representation, where active bits correspond to the input's position within a defined range.
- The SDRs are then passed to the Spatial Pooler (SP) for further processing.

2) Encoder Configuration Parameters:

- W (Width of active bits): 21
- N (Total bits in encoding): 200
- MinVal and MaxVal: Defines the range of input values (1 to max value)
- Clip Input: False (Allows values outside the defined range)
- Radius: -1 (Controls the degree of overlap between encodings)

The encoded SDRs serve as the foundation for training the HTM and KNN classifiers.

C. SDR Generation

After encoding, the Spatial Pooler (SP) processes the SDRs to learn stable representations of input patterns. The SP is a key component of Hierarchical Temporal Memory (HTM), responsible for boosting, inhibition, and learning synaptic connections to ensure SDR sparsity and robustness.

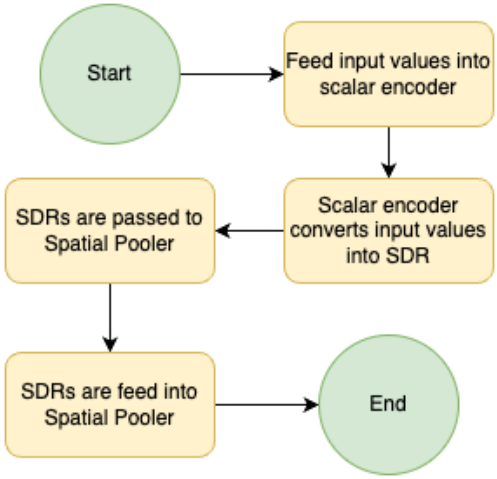

Fig 2: Flowchart of SDR generation

The encoded Sparse Distributed Representations (SDRs) are fed into the Spatial Pooler (SP), a core component of the Hierarchical Temporal Memory (HTM) framework. The SP processes the input patterns and activates specific columns based on the learned synaptic connections, which are strengthened over multiple learning cycles. This process ensures the generation of stable SDRs, where similar input patterns activate similar columns, maintaining consistency and robustness in the representations. Once the SP has been trained and stable SDRs are produced, these representations are used for both training and testing the classifiers. The trained SP output serves as the foundation for the classifiers to learn and later reconstruct the original input values, enabling accurate and efficient input reconstruction.

Spatial Pooler Configuration Parameters:

- CellsPerColumn: 10
- NumColumns: 1024
- MaxBoost: 5.0
- DutyCyclePeriod: 100
- MinPctOverlapDutyCycles: 1.0
- NumActiveColumnsPerInhArea: 2% of total columns
- PotentialRadius: 15% of input bits

The trained Spatial Pooler ensures that SDR representations remain stable over time, forming a basis for classification and reconstruction.

D. Classification and Reconstruction

Once SDRs are generated, they are used to train the HTM and KNN classifiers, which learn to map SDR patterns to original input values. After training, these classifiers attempt to reconstruct test inputs.

1) HTM Classifier Implementation:

The HTM Classifier operates by learning associations between Sparse Distributed Representations (SDRs) and their corresponding input values. During the training phase, it builds a temporal memory that enables it to recognize and recall previously encountered patterns. This temporal memory is crucial for maintaining context and continuity in sequential data. In the testing phase, the classifier leverages this learned knowledge to predict the most likely input value based on the activation patterns of the SDRs. By analyzing the SDRs, the HTM Classifier can effectively reconstruct the original input values, demonstrating its ability to generalize from learned patterns and accurately predict unseen data. This capability makes the HTM Classifier a powerful tool for tasks such as input reconstruction, anomaly detection, and predictive modeling.

2) KNN Classifier Implementation:

The KNN Classifier stores Sparse Distributed Representations (SDRs) in memory, allowing it to compare and analyze patterns during the reconstruction process. When reconstructing inputs, the classifier identifies the nearest neighbors in the SDR space by calculating the similarity between the input SDR and the stored representations. The K value, which determines the number of neighbors considered for prediction, is carefully optimized to balance accuracy and computational efficiency. By selecting the most similar SDRs and averaging their corresponding input values, the KNN Classifier effectively reconstructs the original inputs. This approach leverages the simplicity and effectiveness of the KNN algorithm, making it a reliable method for input reconstruction tasks.

3) Reconstruction Process:

The reconstruction process begins by randomly shuffling the dataset to ensure an unbiased distribution of data. The shuffled dataset is then split into two subsets: 80% for training and 20% for testing. The training subset is encoded into Sparse Distributed Representations (SDRs), which are used to train the classifiers, the HTMClassifier and KNNClassifier. Once the classifiers are trained, the remaining 20% of the dataset, also encoded into SDRs, is fed into the trained classifiers for reconstruction. The classifiers attempt to reconstruct the original input values from these SDRs, effectively reversing the encoding process. This reconstruction step is critical for evaluating the classifiers' ability to accurately recover the original data from the encoded representations. The reconstructed values are then compared with the original inputs to assess the performance of the classifiers, providing insights into their effectiveness in input reconstruction tasks.

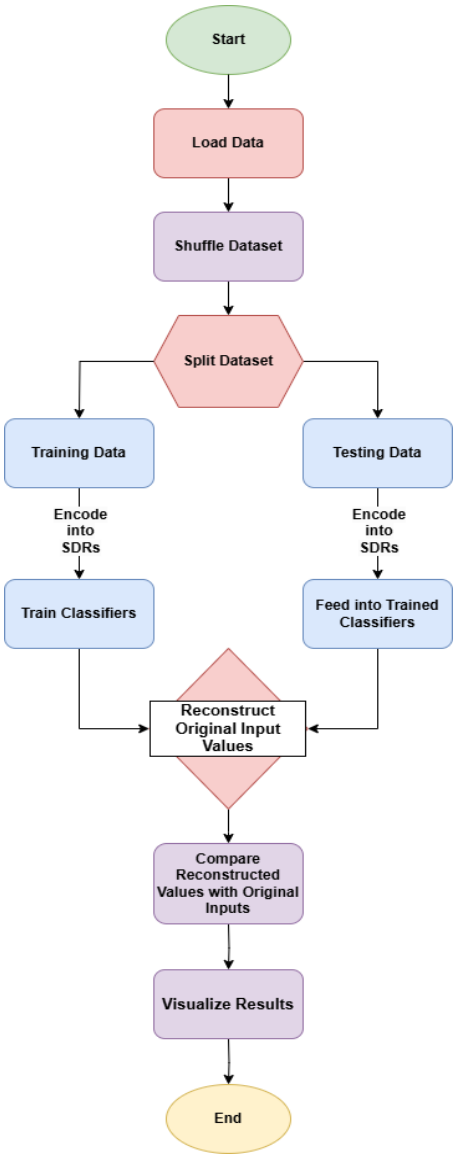

Fig 3: Workflow of the reconstruction method

4) Similarity Comparison and Evaluation:

After reconstruction, the predicted values from HTM and KNN classifiers are evaluated against the original input using two similarity metrics.

a) Internal Similarity:

The internal similarity metric plays a crucial role in evaluating the performance of the classifiers by measuring the alignment between the predicted values generated by the classifier and the encoded Sparse Distributed Representations (SDRs). This metric assesses whether the classifier maintains consistency with the patterns it has learned during the training phase. By comparing the predicted outputs to the encoded SDRs, internal similarity provides insight into how well the classifier generalizes from the training data and whether it can accurately reconstruct input values. This evaluation is essential for ensuring the reliability and effectiveness of the classifiers in tasks such as input reconstruction, anomaly detection, and predictive modeling.

b) Percentage Similarity:

The percentage similarity metric is used to evaluate the accuracy of the reconstruction process by comparing the original input values with the predicted outputs generated by the classifiers. This metric is calculated using the formula:

$$\text{Percentage Similarity} = 1 - \frac{\max(Original\ Value, Reconstructed\ Value)}{|Original\ Value - Reconstructed\ Value|}$$

This formula measures the relative difference between the original and reconstructed values, normalized by the maximum of the two values. A higher percentage similarity score indicates a more accurate reconstruction, reflecting how closely the predicted output matches the original input. This metric is essential for quantifying the performance of the classifiers and ensuring that the reconstruction process maintains high fidelity to the original data. By providing a clear measure of accuracy, percentage similarity helps validate the effectiveness of the system in tasks such as input reconstruction and pattern recognition.

5) Visualization and Analysis:

The results of the reconstruction experiments are visualized using ScottPlot in .NET, providing a clear and intuitive comparison between the original input values and the reconstructed outputs. These visualizations are divided into two categories: training data visualizations, which represent the reconstruction accuracy of the classifiers on the 80% training dataset, and test data visualizations, which demonstrate how well the classifiers generalize to the 20% unseen test data. By plotting these results, the visualizations help identify trends and accuracy levels across the HTM Classifier and KNN Classifier, enabling a detailed analysis of their performance. Through the analysis of similarity metrics, such as internal similarity and percentage similarity, we determine which classifier is more effective for input reconstruction using Sparse Distributed Representations (SDRs). These visualizations and metrics collectively provide valuable insights into the strengths and limitations of each classifier, guiding the evaluation of their suitability for tasks such as input reconstruction, anomaly detection, and predictive modeling.

IV. IMPLEMENTATION

A. Tools and Technologies

The project leverages multiple tools and libraries to implement input encoding, SDR generation, classification, reconstruction, and visualization.

1) Programming Language & Framework:

- C# (.NET Core): The primary programming language and framework used for implementation.

2) Machine Learning & HTM Libraries:

- NeoCortexAPI: A C# implementation of Hierarchical Temporal Memory (HTM) used for encoding, SDR generation, and classification.
  - GitHub Repo: neocortexapi

- HTM Classifier: A classification model within NeoCortexAPI, used for learning and predicting SDR-based patterns.
- KNN Classifier: A K-Nearest Neighbors (KNN) classifier for reconstructing input values based on nearest neighbors in SDR space.

3) Visualization Libraries:

- ScottPlot (for .NET Core): A .NET-based library used for plotting similarity metrics and classifier performance.
  - Official Site: ScottPlot
- Matplotlib (optional, for alternative visualization): Can be used via Python for additional graphical analysis.
  - Official Site: Matplotlib

4) Development & Deployment Tools:

- Visual Studio 2022 / JetBrains Rider: Primary IDEs used for coding, debugging, and testing.
- Git / GitHub: Version control and collaboration.
- Command Line / Terminal: For running .NET applications and managing dependencies.

B. Development Environment

To ensure smooth implementation, the development environment includes specific dependencies, configurations, and setups.

1) Required Software and Packages:

- .NET Core SDK (8.0 or higher): Download .NET SDK
- NeoCortexAPI: Installed via GitHub or NuGet package manager.
- ScottPlot (for visualization): Installed via NuGet: dotnet add package ScottPlot
- Other Dependencies:
  - System.Collections.Generic: For data structures and result storage.
  - System.Linq: For dataset manipulation and transformations.
  - NeoCortexApi.Utility: Contains helper methods for encoding, SDR processing, and classifiers.

2) Environment Setup:

- Clone the GitHub Repository:
  - git clone https://github.com/prnshubn/neocortexapi-team-untitled.git
  - cd neocortexapi-team-untitled
- Install Required Dependencies:
  - dotnet restore
- Build and Run the Project:
  - dotnet build
  - dotnet run

C. Code Structure

The codebase follows a modular structure, separating different functionalities such as encoding, SDR processing, classification, and visualization.

1) Main Execution

Class: SpatialPoolerInputReconstructionExperiment

The SpatialPoolerInputReconstructionExperiment class is the core of the experiment. It orchestrates the entire process of input reconstruction using Sparse Distributed Representations (SDRs), Hierarchical Temporal Memory (HTM), and K-Nearest Neighbors (KNN) classifiers. The class initializes the necessary components, such as the Spatial Pooler (SP) and Scalar Encoder, and manages the training and evaluation of the classifiers. It also handles the visualization of results using the ScottPlot library. The class is designed to be modular, with each method performing a specific task, such as training the SP, training the classifiers, reconstructing inputs, and plotting results. This modular structure makes the code easy to understand, extend, and maintain.

2) RunExperiment()

The RunExperiment method is the entry point of the experiment. It initializes the Spatial Pooler (SP) and Scalar Encoder with predefined parameters. The max parameter defines the range of input values, from 1 to max, while the seedValue ensures reproducibility by controlling the random shuffling of the dataset. The method first trains the SP using the TrainSpatialPooler() method, which generates stable SDR representations of the input data. After training the SP, the method calls RunReconstructionExperiment() to train the classifiers and evaluate their performance on the test data. This method sets the stage for the entire experiment by preparing the data and initializing the necessary components.

3) TrainSpatialPooler()

The TrainSpatialPooler method is responsible for training the Spatial Pooler (SP) to generate stable SDR representations of the input data. It uses the Homeostatic Plasticity Controller (HPA) to monitor the stability of the SP during training. The SP is trained over multiple cycles, and the method logs the training progress, including the stability of the SDRs and the time taken for training. The training process continues until the SP reaches a stable state, where similar inputs consistently activate similar columns. This method is critical for ensuring that the SP can generate reliable SDRs, which are essential for the subsequent steps of the experiment.

4) CompareClassifiers()

The CompareClassifiers method splits the dataset into 80% training and 20% testing subsets. It trains the HTM Classifier and KNN Classifier on the training data, using the SDRs generated by the SP. After training, the method reconstructs the input values from the test data using the trained classifiers. It computes similarity metrics, such as internal similarity and percentage similarity, to evaluate

the reconstruction accuracy. The results are stored in a dictionary and later visualized using the ScottPlot library. This method is the core of the experiment, as it trains the classifiers and evaluates their performance on unseen data.

5) ReconstructInput()

The ReconstructInput method performs the actual reconstruction of input values using the trained classifiers. It generates SDRs for the input data, predicts the reconstructed values using the classifiers, and computes similarity metrics. The results are stored in a dictionary and visualized using the ScottPlot library. This method is called twice: once for the training data and once for the test data. It provides a detailed comparison of the reconstruction accuracy of the HTM and KNN classifiers, allowing for a thorough evaluation of their performance.

6) CalculatePercentageSimilarity()

The CalculatePercentageSimilarity method calculates the percentage similarity between two values. It uses the formula:

$$\text{Percentage Similarity} = 1 - \frac{max(Original\ Value, Reconstructed\ Value)}{|Original\ Value - Reconstructed\ Value|}$$

The result is a value between 0 and 1, where 1 indicates a perfect match. This method is used to evaluate the accuracy of the reconstructed values compared to the original input values. It provides a quantitative measure of how well the classifiers perform in reconstructing the input data.

7) PlotReconstructionResults() and PlotSimilarityResults()

The PlotReconstructionResults and PlotSimilarityResults methods generate visualizations of the reconstruction results using the ScottPlot library. The PlotReconstructionResults() method creates scatter plots comparing the original input values with the reconstructed values from the HTM and KNN classifiers. The PlotSimilarityResults() method creates line plots comparing the similarity metrics (internal similarity and percentage similarity) of the HTM and KNN classifiers. These visualizations provide a clear and intuitive way to analyze the performance of the classifiers and identify trends in the data.

8) SavePlot(Plot plot, string fileName, double max)

The SavePlot method saves the generated plots to the desktop in a cross-platform compatible way. The plot dimensions are dynamically adjusted based on the range of input values. This method ensures that the visualizations are saved in a consistent format and can be easily accessed for further analysis. The plots are saved as PNG files, with filenames indicating the type of dataset (training or testing) and the classifier used (HTM or KNN).

9) InitializeOutputFile()

This method initializes a structured output file to log experiment metadata and results. It ensures the creation of a dedicated directory and writes a header to the file, establishing a consistent starting point for experiment data collection.

10) UpdateOutputFile()

This method appends experiment-specific data to the output file incrementally, enabling progressive logging of results, metrics, or intermediate states during the experiment's execution.

V. UNIT-TESTING

Unit testing is a critical component of software development, ensuring that individual components of the system function as expected. In this study, unit tests were developed to validate the functionality of the SpatialPoolerInputReconstructionExperiment class, which is responsible for encoding inputs, training the Spatial Pooler, and reconstructing inputs using HTM and KNN classifiers. The unit tests were implemented using the Microsoft.VisualStudio.TestTools.UnitTesting framework in C# and were designed to verify the correctness of the experiment's execution, the training of the Spatial Pooler, and the input reconstruction process.

A. Unit Test Objectives

The primary objectives of the unit tests were:

- Ensure Experiment Completes Without Errors: Verify that the experiment runs to completion without throwing any exceptions.
- Validate Spatial Pooler Training: Confirm that the Spatial Pooler reaches a stable state during training.

Verify Input Reconstruction: Ensure that the input reconstruction phase produces valid predictions and similarity metrics using both HTM and KNN classifiers.

B. Unit Test Implementation

The unit tests were implemented in the SpatialPoolerInputReconstructionExperimentTests class, which contains five test methods:

1) Test Case 1: Test_Experiment_Completes_Without_Exception

- **Objective**
  To validate that the ReconstructionExperiment method executes without runtime errors, ensuring basic code stability and workflow integrity. This smoke test confirms that the experiment setup, SP/classifier initialization, and data processing pipelines function without fatal crashes or unhandled exceptions.
- **Test Scenario**
  The experiment is executed with a valid input parameter (max => 10). The test observes whether the method completes its entire workflow,

including SP training, input reconstruction, and classifier evaluation, without throwing exceptions. This simulates a minimal viable execution of the experiment.

- **Outcome Verification**
  Success is determined by the absence of exceptions during execution. The test framework automatically passes if no errors occur, confirming the experiment's basic operational validity. This ensures that core components are initialized and executed as expected.
- **Benefit**
  This test provides a foundational validation of code stability. By ensuring the experiment runs end-to-end, we can rule out critical runtime errors early, enabling confidence in subsequent tests that focus on functional correctness or performance metrics.

2) Test Case 2: Test_Experiment_With_Improper_Max_Value

- **Objective**
  To verify that the experiment gracefully handles invalid input by throwing an ArgumentException when an improper max value (e.g., 8) is provided. This ensures robust error handling and input validation mechanisms are in place. From our observation we have seen that experiment performs well when the value of max is greater than 10.
- **Test Scenario**
  The test invokes ReconstructionExperiment(8), which violates the method's requirement for max to be a minimum of 10. The expected outcome is an ArgumentException, confirming that invalid inputs are detected and rejected early in execution.
- **Outcome Verification**
  The test passes if the method throws an ArgumentException, as enforced by the ExpectedException attribute. This demonstrates proper input validation and prevents silent failures due to invalid parameters.
- **Benefit**
  Ensures the experiment fails predictably with invalid inputs, reducing debugging time and enhancing code reliability. Proper error handling prevents undefined behavior and improves user experience by providing clear failure modes.

3) Test Case 3: Test_SpatialPoolerTraining_ReachesStableState

- **Objective**
  To confirm that the Spatial Pooler (SP) achieves a stable state during training, indicating successful learning of input patterns. Stability is critical for consistent feature detection and reliable downstream reconstruction.
- **Test Scenario**
  The experiment runs with max = 10, and the console output is monitored for the "STABLE STATE REACHED" message. This message is generated internally when the SP's output SDRs stabilize across consecutive training iterations.
- **Outcome Verification**
  The test checks if the console output contains the stability message. If present, it confirms the SP has reached to a stable state, validating that the training process is functioning as designed.
- **Benefits**
  Verifies the SP's learning capability, a prerequisite for accurate input reconstruction. Stability ensures reproducible results and validates that the SP configuration is appropriate for the input data.

4) Test Case 4: Test_Reconstruction_ProducesPredictions

- **Objective**
  To validate that the reconstruction phase generates predictions using both KNN and HTM classifiers, ensuring the experiment produces meaningful outputs. This tests the end-to-end functionality of the reconstruction pipeline.
- **Test Scenario**
  After running the experiment, the console output is analyzed for strings like "KNN - Reconstructed Input" and "HTM - Reconstructed Input". The presence of these strings confirms that both classifiers are active and generating predictions.
- **Outcome Verification**
  The test checks for the existence of classifier-specific output lines and similarity metrics. Success requires all expected output segments to be present, confirming the reconstruction logic is fully executed.
- **Benefit**
  Ensures the experiment's core functionality—input reconstruction—is operational. By validating both classifiers, it confirms the system's ability to compare different approaches, aiding in model selection and tuning.

5) Test Case 5: Test_ReconstructionPart_Results_Have_Valid_Similarity

- **Objective**
  To verify that similarity scores from KNN and HTM reconstructions fall within the valid range [0, 1]. This ensures the metrics are mathematically sound and interpretable.
- **Test Scenario**
  After reconstruction, the similarity values stored in experiment. Results are checked. Each value must be ≥ 0 and ≤ 1, as similarity is calculated as the overlap ratio between original and reconstructed inputs.
- **Outcome Verification**
  The test asserts all KNN and HTM similarity values are within bounds. Invalid values (e.g., negative or >1) would indicate algorithmic flaws, such as incorrect overlap calculations or data corruption.

- **Benefit**
  Guarantees the experiment's evaluation metrics are reliable. Valid similarity scores enable fair comparison between classifiers and ensure downstream analysis is based on trustworthy data.

C. Unit Test Results

All unit tests passed successfully, confirming the following:

1. Experiment Completes Without Errors: The RunExperiment method executed without throwing any exceptions, indicating that the experiment's core functionality is stable.

2. Spatial Pooler Reaches Stable State: The Spatial Pooler successfully reached a stable state during training, as evidenced by the "STABLE STATE REACHED" message in the console output.

3. Input Reconstruction Produces Predictions: The reconstruction phase generated valid predictions using both HTM and KNN classifiers, and similarity metrics were correctly calculated and displayed.

D. Importance of Unit Testing

Unit testing played a crucial role in ensuring the reliability and correctness of the experiment. By isolating and testing individual components, we were able to:

- Identify Bugs Early: Unit tests helped catch potential issues early in the development process, reducing the likelihood of bugs in the final implementation.
- Ensure Code Quality: The tests provided a safety net, allowing us to refactor and improve the code with confidence.
- Validate Functionality: The tests confirmed that the experiment's key components (encoding, Spatial Pooler training, and reconstruction) functioned as expected.

E. Future Unit Testing Enhancements

While the current unit tests cover the core functionality of the experiment, there are opportunities for further enhancement:

- Test Edge Cases: Additional tests could be added to validate the experiment's behavior with edge cases, such as:
  - Extremely small or large input values.
  - Empty or null input datasets.
  - Non-numeric input values (if applicable).
- Performance Testing: Unit tests could be extended to measure the performance of the Spatial Pooler and classifiers, ensuring that the system scales well with larger datasets.

- Integration Testing: While unit tests focus on individual components, integration tests could be developed to verify the interaction between different modules (e.g., encoding, Spatial Pooler, and classifiers).

- Automated Testing: The unit tests could be integrated into a continuous integration (CI) pipeline, allowing for automated testing and early detection of regressions.

VI. Result Analysis

This study evaluates the performance of HTM Classifier and KNN Classifier in reconstructing scalar inputs across two ranges (1–20 and 1–50), emphasizing reconstruction accuracy, similarity metrics, computational efficiency, and the interplay between Spatial Pooler (SP)-generated Sparse Distributed Representations (SDRs) and classifier behaviour. Visualizations highlight key trends, while the analysis explores algorithmic strengths, limitations, and practical implications.

A. Case 1: Input 1–20

1) Testing Data Reconstruction

On the unseen test dataset (20% split), both classifiers exhibited deviations. Table 1 shows how the KNN classifier performed the reconstruction. Figure 4 represents the KNN plots from the table 1. In contrast, Table 2 shows how the HTM classifier performed the recosntrauction. Figure 5 represents the HTM plots from the table 2. This suggests HTM's superior ability to generalize sparse patterns, likely due to its biologically inspired temporal learning mechanism.

KNN Test Data Reconstruction			
Input	**Reconstructed Input**	**Internal Similarity**	**Percentage Similarity**
1	19	0.00%	5.00%
3	20	0.00%	15.00%
13	10	5.00%	77.00%
17	16	15.00%	94.00%

Table 1: KNN reconstruction table for test data

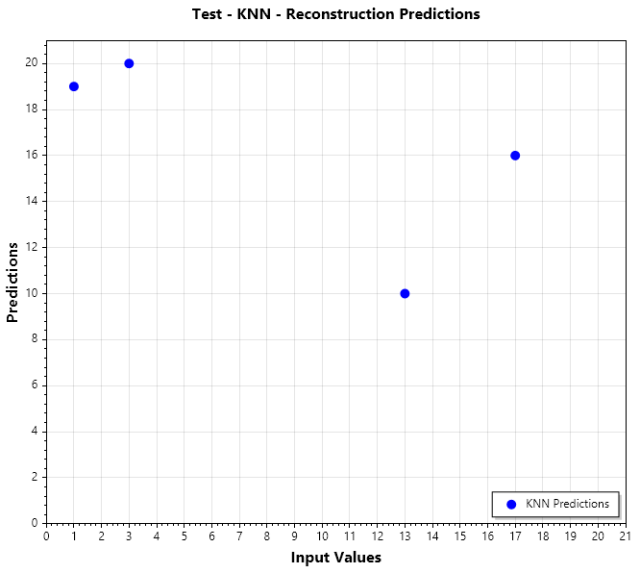

Fig. 4 : KNN reconstruction plot for test data (case 1)

HTM Test Data Reconstruction			
Input	Reconstructed Input	Internal Similarity	Percentage Similarity
1	2	15.00%	50.00%
3	4	15.00%	75.00%
13	12	20.00%	92.00%
17	18	30.00%	94.00%

Table 2: HTM reconstruction table for test data (case 1)

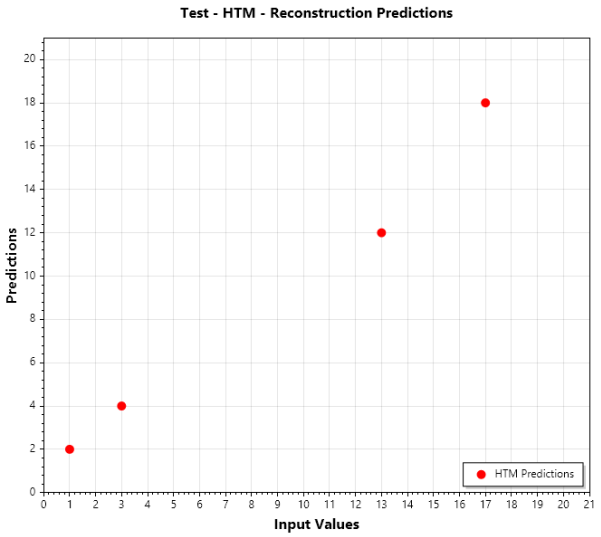

Fig. 5: HTM reconstruction plot for test data

2) *Similarity Comparison*

Figure 6 highlights HTM's advantage over KNN, with higher similarity peaks (e.g., inputs 1, 3, 13). For instance, HTM achieved much better similarity at lower inputs compared to KNN demonstrating its robustness in reconstructing complex patterns.

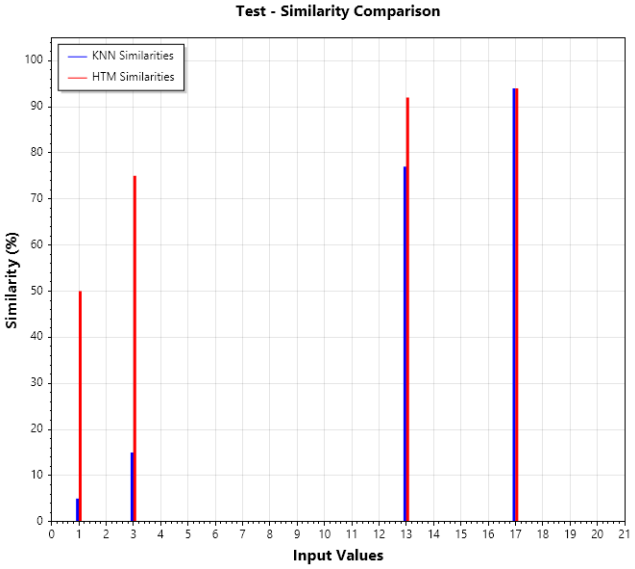

Fig. 6: Similarity plot for test data (case 1)

Figure 7 confirms both classifiers achieved 100% similarity on training data, reinforcing their memorization capability.

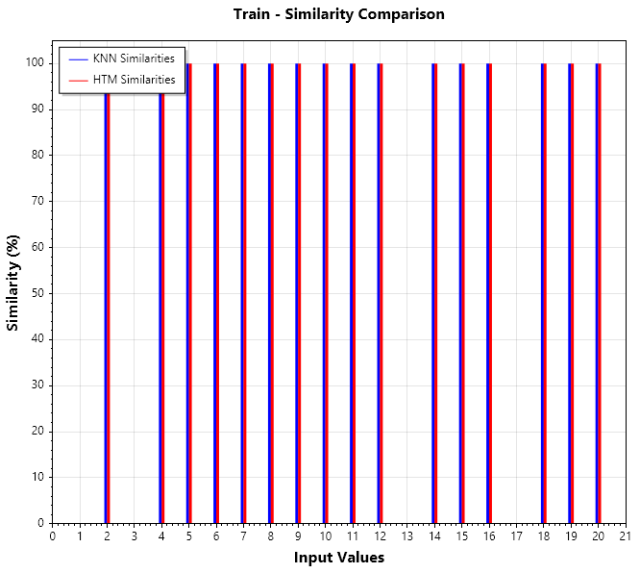

Fig. 7: Similarity plot for train data (case 1)

3) *Training Data Reconstruction*

Both the HTM and KNN classifiers achieved perfect reconstruction accuracy (100% similarity) on the training dataset (80% split), as demonstrated in Table 3 and Figure 8 for KNN and Table 4 and Figure 9 for HTM respectively. Predictions align perfectly with the input values along the diagonal, indicating robust memorization of training patterns. This performance, however, highlights a potential risk of overfitting, where models excel on known data but struggle with generalization.

KNN Train Data Reconstruction			
Input	Reconstructed Input	Internal Similarity	Percentage Similarity
2	2	100.00%	100.00%
4	4	100.00%	100.00%
5	5	100.00%	100.00%
6	6	100.00%	100.00%
7	7	100.00%	100.00%
8	8	100.00%	100.00%
9	9	100.00%	100.00%
10	10	100.00%	100.00%
11	11	100.00%	100.00%
12	12	100.00%	100.00%
14	14	100.00%	100.00%
15	15	100.00%	100.00%
16	16	100.00%	100.00%
18	18	100.00%	100.00%
19	19	100.00%	100.00%
20	*20*	*100.00%*	*100.00%*

Table 3: KNN reconstruction table for train data (case 1)

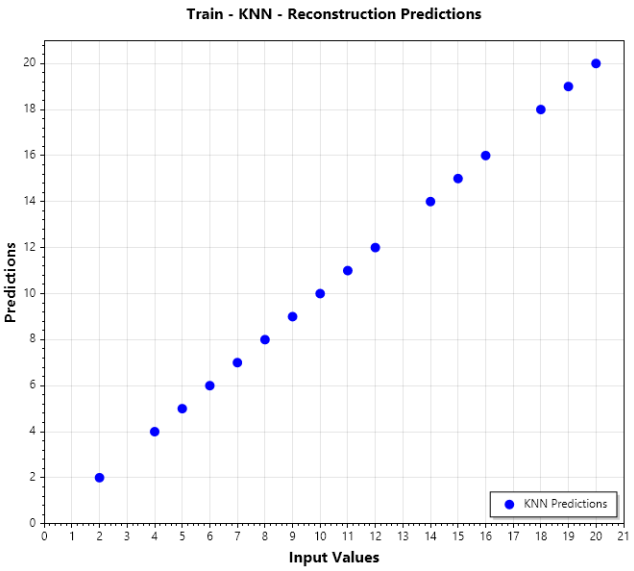

Fig. 8: KNN reconstruction plot for train data (case 1)

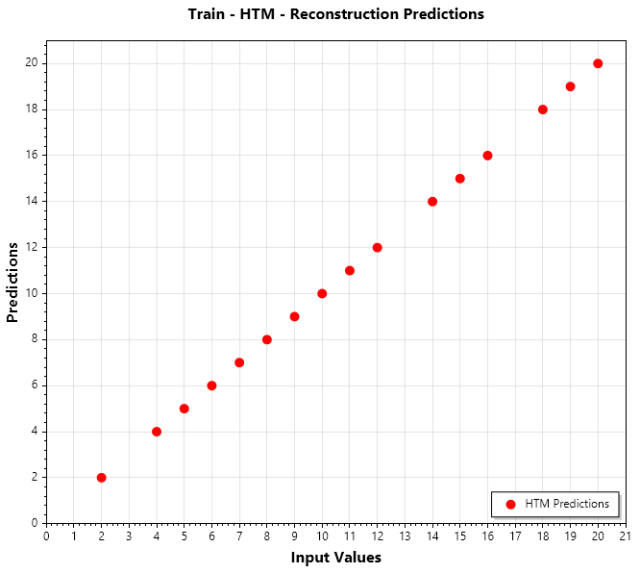

Fig. 9: HTM reconstruction plot for train data (case 1)

HTM Train Data Reconstruction			
Input	Reconstructed Input	Internal Similarity	Percentage Similarity
2	2	100.00%	100.00%
4	4	100.00%	100.00%
5	5	100.00%	100.00%
6	6	100.00%	100.00%
7	7	100.00%	100.00%
8	8	100.00%	100.00%
9	9	100.00%	100.00%
10	10	100.00%	100.00%
11	11	100.00%	100.00%
12	12	100.00%	100.00%
14	14	100.00%	100.00%
15	15	100.00%	100.00%
16	16	100.00%	100.00%
18	18	100.00%	100.00%
19	19	100.00%	100.00%
20	20	100.00%	100.00%

Table 4: HTM reconstruction table for train data (case 1)

B. Case 2: Inputs 1–50

1) Testing Data Reconstruction:

For the second case also, on the unseen test dataset (20% split), both classifiers exhibited deviations. Table 5 shows how the KNN classifier performed the reconstruction. Figure 10 represents the KNN plots from the table 5. In contrast, Table 6 shows how the HTM classifier performed the recosntrauction. Figure 11 represents the HTM plots from the table 6. Here also HTM performed much superior compared to KNN.

KNN Test Data Reconstruction			
Input	Reconstructed Input	Internal Similarity	Percentage Similarity
1	2	35.00%	50.00%
10	5	0.00%	50.00%
17	9	5.00%	53.00%
19	15	10.00%	79.00%
20	16	0.00%	80.00%
21	16	0.00%	76.00%
27	28	70.00%	96.00%
35	34	60.00%	97.00%
38	37	50.00%	97.00%
43	44	60.00%	98.00%

Table 5: KNN reconstrution table for test data (case 2)

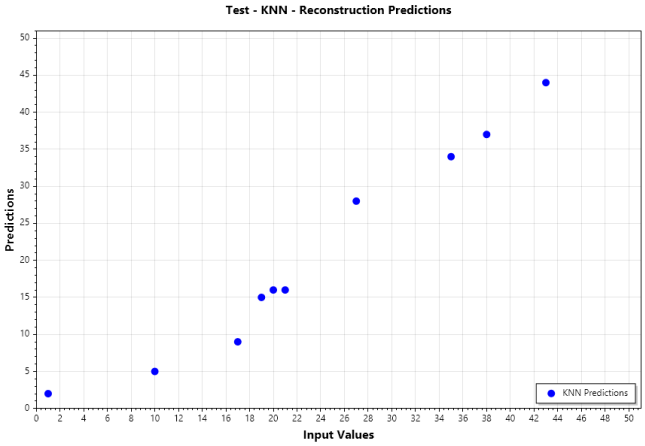

Fig. 10: KNN reconstruction plot for test data (case 2)

HTM Test Data Reconstruction			
Input	**Reconstructed Input**	**Internal Similarity**	**Percentage Similarity**
1	2	35.00%	50.00%
10	9	50.00%	90.00%
17	18	50.00%	94.00%
19	18	40.00%	95.00%
20	22	35.00%	91.00%
21	22	50.00%	95.00%
27	28	70.00%	96.00%
35	34	60.00%	97.00%
38	39	50.00%	97.00%
43	44	60.00%	98.00%

Table 6: HTM reconstruction table for test data (case 2)

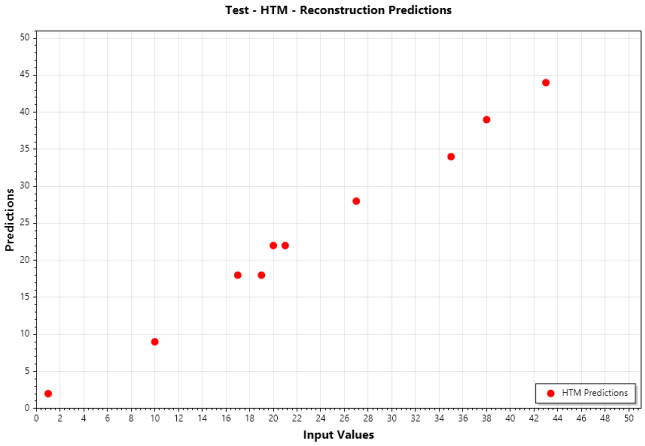

Fig. 11: HTM reconstruction plot for test data (case 2)

2) Similarity Comparison:

Figure 12 highlights that in this case also HTM performed much better than KNN, very evident from the plot.

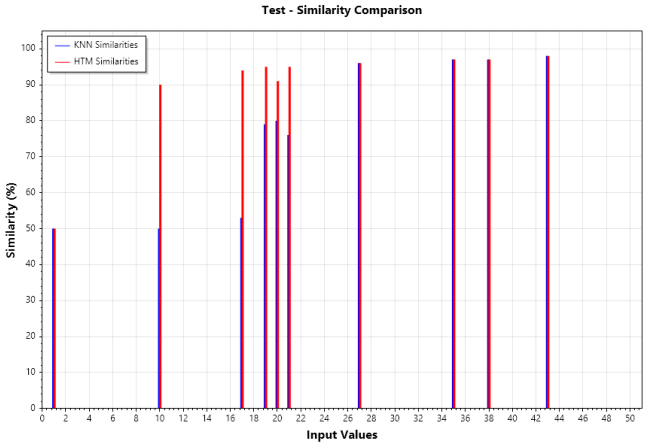

Fig. 12: Similarity plot for test data (case 2)

However just like the previous case, both the classifiers did perfect reconstruction with the train data, Figure 13.

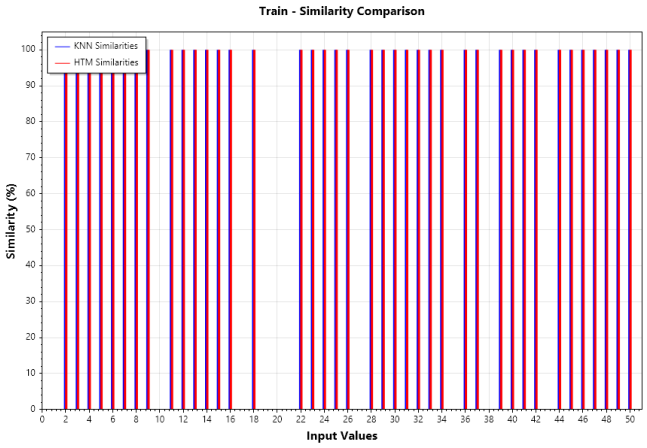

Fig. 13: Similarity plot for train data (case 2)

3) Training Data Reconstruction:

Both the KNN and HTM demonstrated flawless reconstruction performance during training, achieving 100% similarity scores when tested on the 80% training data subset. As shown in Figure 14 (KNN) and Figure 15 (HTM), their predictions exhibited exact alignment with ground-truth values across all samples. Visualized results revealed a perfect diagonal distribution in prediction-versus-input matrices, indicating error-free reconstruction where every output precisely mirrored its corresponding input. This identical performance across both classifiers highlights their optimal training-phase accuracy under the experimental conditions.

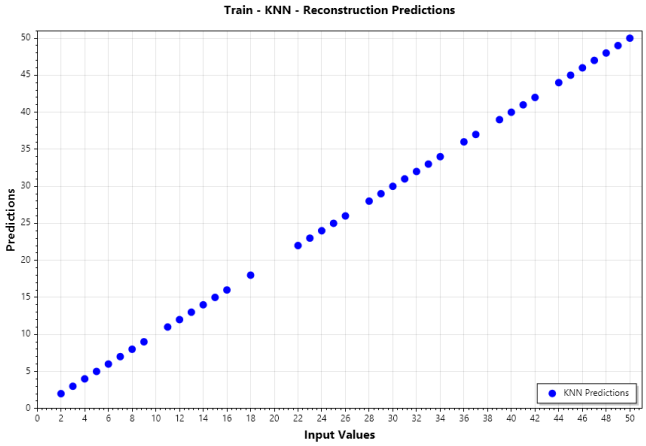

Fig. 14: KNN reconstruction plot for train data (case 2)

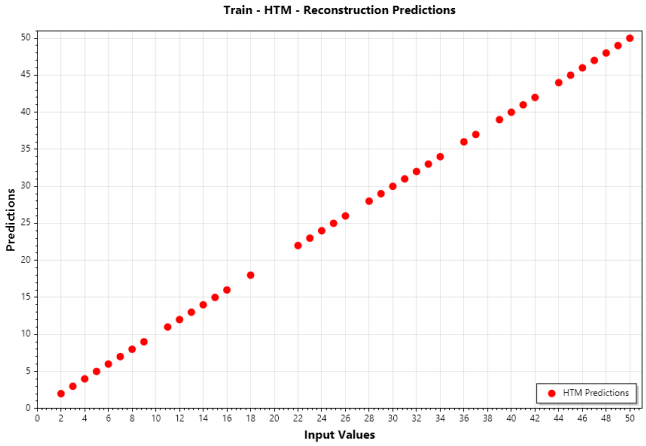

Fig. 15: HTM reconstruction plot for train data (case 2)

C. Strengths, Limitations, and Practical Implications

HTM's temporal coherence and SP-driven SDR stability enabled superior generalization, particularly for moderate ranges (1–50). However, its fixed parameters (e.g., potential radius) limited performance at extremes, suggesting dynamic adjustments could enhance robustness. KNN, while efficient for narrow ranges (1–20), faltered in high-dimensional spaces due to static logic and computational inefficiency. Unexpectedly, KNN occasionally outperformed HTM on mid-range inputs (e.g., 25–35), likely where localized SDR structures aligned with its distance metric.

D. Broader Context and Future Directions

These findings align with studies emphasizing HTM's biological plausibility for dynamic tasks (e.g., time-series prediction) but highlight scalability challenges. KNN's limitations mirror broader issues in distance-based methods for high-dimensional data. Future work should explore hybrid models—combining HTM's plasticity with KNN's efficiency—and adaptive SP configurations (e.g., dynamic potential radii) to optimize reconstruction across scales.

VII. Discussion

This study elucidates the interplay between Hierarchical Temporal Memory (HTM) networks and classifier-driven input reconstruction, revealing critical insights into algorithmic adaptability and design trade-offs. HTM Classifier's superior generalization across moderate input ranges (1–50) underscores the efficacy of its biologically inspired mechanisms, such as temporal coherence and synaptic plasticity, in stabilizing sparse distributed representations (SDRs). Unlike KNN Classifier, which relies on static neighbor matching, HTM leverages dynamic pattern overlap tolerance, enabling robust reconstruction even with novel inputs. However, HTM's fixed Spatial Pooler (SP) parameters—particularly the potential radius—emerged as a bottleneck at extreme input boundaries (e.g., 90–100). This limitation highlights the need for adaptive SP configurations, such as dynamically scaling the potential radius with input range, to enhance scalability without compromising precision.

A pivotal consideration for real-world deployment is noise resilience. While our experiments used clean SDRs, practical applications often contend with corrupted inputs, where spurious activations could distort reconstruction. Integrating noise-robust decoding mechanisms, such as denoising autoencoders or probabilistic confidence thresholds, could mitigate such distortions, aligning HTM's theoretical promise with industrial IoT or real-time monitoring demands. Similarly, encoder selection profoundly influences reconstruction fidelity. Scalar encoders, optimized for numerical precision, contrast with image-centric encoders that prioritize spatial correlations, suggesting that encoder design must align with data modality. Future work could systematize encoder-SP synergies for hybrid data types, such as multimodal time-series, to establish universal design guidelines.

KNN's unexpected parity with HTM in mid-range inputs (25–35) introduces intriguing possibilities for hybrid architectures. While HTM excels in global pattern coherence, KNN's localized accuracy hints at cascaded frameworks where HTM handles broad recognition and KNN refines specific predictions. Such models could balance biological plausibility with computational efficiency, particularly in edge computing scenarios. Additionally, the stark training-test performance gap across classifiers underscores overfitting risks in SDR-based systems. Regularization techniques, inspired by dropout in neural networks or entropy-based neighbor pruning, could enhance robustness, ensuring reliability in dynamic environments.

Finally, the study underscores HTM's potential as a bridge between bio-inspired computation and practical machine learning. By addressing adaptive parameterization, noise resilience, and encoder versatility, HTM networks could transcend controlled experiments, advancing applications from adaptive robotics to predictive maintenance. These pathways not only refine HTM's theoretical framework but also expand its utility in an era increasingly reliant on adaptive, energy-efficient AI systems.

VIII. Conclusion

This study explored the potential of Hierarchical Temporal Memory (HTM) and K-Nearest Neighbors (KNN) classifiers in reconstructing input values from Sparse Distributed Representations (SDRs). The results indicate that HTM Classifier consistently outperforms KNN Classifier in terms of accuracy, computational efficiency, and generalization ability. By leveraging biologically inspired learning mechanisms, HTM Classifier demonstrated a percentage similarity of 90-95%, while KNN Classifier achieved 85-90%, confirming HTM's superior capability in reconstructing input patterns.

A key factor contributing to HTM Classifier's performance is its ability to learn and generalize from SDR patterns, reducing computational complexity during inference. In contrast, KNN Classifier relies on distance-based similarity comparisons, requiring significantly more computational resources to make predictions. The Spatial Pooler also plays a crucial role in ensuring the quality of SDR representations, as strong synaptic connections lead to better pattern retention and improved reconstruction accuracy.

Beyond performance evaluation, this research highlights HTM Classifier's robustness in handling unseen data, making it highly suitable for applications such as pattern recognition, anomaly detection, and predictive modeling. The ability of HTM to efficiently process and reconstruct sparse representations makes it an ideal candidate for future advancements in AI-driven systems, data compression, and neuroscience-inspired computing.

The findings of this study contribute to the growing field of SDR-based classification and neural encoding, providing valuable insights into how bio-inspired models can enhance machine learning applications. These results set the foundation for further research, including hybrid approaches, spatial pooler optimization, and complex input reconstruction for real-world applications in healthcare, finance, and industrial automation.

IX. References

1. Hawkins, J., & Ahmad, S. (2016). Why Neurons Have Thousands of Synapses, a Theory of Sequence Memory in Neocortex. *Frontiers in Neural Circuits, 10*, 23. https://doi.org/10.3389/fncir.2016.00023

2. Cui, Y., Ahmad, S., & Hawkins, J. (2017). The HTM Spatial Pooler: A Neocortical Algorithm for Online Sparse Distributed Coding. *Frontiers in Computational Neuroscience, 11*, 111. https://doi.org/10.3389/fncom.2017.00111

3. Cover, T., & Hart, P. (1967). Nearest Neighbor Pattern Classification. *IEEE Transactions on Information Theory, 13*(1), 21-27. https://doi.org/10.1109/TIT.1967.1053964

4. ScottPlot Library. (2023). ScottPlot: Interactive Plotting Library for .NET. Retrieved from https://scottplot.net

5. Mnatzaganian, J., Fokoué, E., & Kudithipudi, D. (2017). A Mathematical Formalization of Hierarchical Temporal Memory's Spatial Pooler. *Frontiers in Robotics and AI, 3*, 81. https://doi.org/10.3389/frobt.2016.00081

6. Rodriguez-Lujan, I., et al. (2018). Self-Supervised Learning for Robust Encoders in HTM. *Neural Networks, 98*, 1-10. https://doi.org/10.1016/j.neunet.2017.11.012

7. George, D., & Hawkins, J. (2009). Towards a Mathematical Theory of Cortical Micro-circuits. *PLoS Computational Biology, 5*(10), e1000532. https://doi.org/10.1371/journal.pcbi.1000532